Formalisme O93_0 – Observation, empreintes, espaces sémantiques, compétences et autonomie

AAM – formalisme normatif versionné

13 octobre 2026

Résumé

Ce document spécifie la conception normative de AAM O93_0 à partir de la théorie principale des empreintes d'ordre 5, de ses compléments, d'O93 et d'O92_0. Le modèle observe des données brutes et des vues multi-sondes, apprend des mécanismes reproductibles et composables, construit un RAG à partir d'espaces sémantiques successifs pondérant itoken/vtoken, et coordonne compétences, mémoire, vérification, feedback et autonomie. Le formalisme ne présume ni réussite ni échec : seuls les résultats testés dans leur protocole sont affirmés.

Table des matières

1 Mission et principes **3**
- 1.1 Mission unifiée . . . 3
- 1.2 Source et priorité . . . 3
- 1.3 Principes normatifs . . . 3

2 Objets fondamentaux **4**
- 2.1 Observation et empreinte . . . 4
- 2.2 Contrat de sonde . . . 4
- 2.3 Distances et collisions . . . 4
- 2.4 Sonde adaptative . . . 4

3 RAG fondé sur les espaces sémantiques **5**
- 3.1 Pipeline canonique . . . 5
- 3.2 Itoken et vtoken . . . 5
- 3.3 Poids finaux et nombre d'espaces . . . 5
- 3.4 Lecteur et contexte de requête . . . 6
- 3.5 Nettoyage et inspection . . . 6

4 Compétences et acquisition **7**
- 4.1 Socle général . . . 7
- 4.2 Définition d'une compétence . . . 7
- 4.3 Acquisition sans fuite . . . 7
- 4.4 Composition et adaptation . . . 7

5 Observateur universel **8**

6 Mémoire, autonomie, causalité **9**
- 6.1 Mémoires . . . 9
- 6.2 Autonomie persistante . . . 9
- 6.3 Causalité des récompenses . . . 9

7 Interface et modes d'exécution **10**
- 7.1 Chat et historique . . . 10
- 7.2 RAG . . . 10
- 7.3 Sondes et compétences . . . 10
- 7.4 Observation et mémoire . . . 10
- 7.5 Entraînement, Internet et réglages . . . 10
- 7.6 Contrat d'observation multimodale et d'ingestion OCR . . . 10

8 Orchestration et pipeline complet **12**

9 Résultats, sécurité et preuve **13**
9.1 Statuts . . . 13
9.2 Sécurité . . . 13
9.3 Format de rapport obligatoire . . . 13

10 Phases et critères de sortie **14**

11 Conséquences et statut **15**
11.1 Théoriques . . . 15
11.2 Conceptuelles . . . 15
11.3 Opérationnelles . . . 15
11.4 Conclusion/statut . . . 15

Chapitre 1

Mission et principes

1.1 Mission unifiée

AAM doit observer un objet accessible par fichier, valeur, flux ou matériel autorisé, produire diverses vues par sondes versionnées, identifier invariants et variantes, construire des empreintes, apprendre ou construire une compétence exécutable, la reproduire et la transférer, l'orchestrer et l'adapter à une situation nouvelle. Toutes les fonctions partagent un contrat de tâche, des statuts, une mémoire de provenance et une vérification indépendante. La recherche lexicale devient une compétence, non le principe fondamental du RAG.

1.2 Source et priorité

Le texte principal est `DDD_Base_theorie_des_empreintes_complete_order_5.tex`, racine et `theorie_emp` ; on utilise aussi les formalismes d'observation et DD_Dev. O93 et O92_0 sont la base logicielle prioritaire. `dataset/` apporte des corpus ; `formalismes/` et `rapports/` apportent l'historique, pas une validation automatique de la version courante.

1.3 Principes normatifs

1. Ne jamais coder ou prédire à l'avance une réussite ou un échec. Les statuts décrivent les résultats effectivement observés.
2. Aucune triche : aucune sortie-cible préécrite, oracle caché ou résultat attendu injecté dans une sonde. Le replay de reproduction s'exécute sans source/oracle après chargement de l'artefact.
3. Observer avant de rechercher, répondre ou apprendre. Les sources ont une provenance ; les sorties récupérées restent non fiables.
4. Unifier chat, RAG, observation, compétences, entraînement, mémoire, autonomie, matériel et réseau dans le même orchestrateur.
5. Garder les états non conclusifs explicites : `NOT_RUN`, `NOT_DECODED`, `PARTIAL`, `UNRESOLVED_BUDGET`, `ENVIRONMENT_ERROR`, `BLOCKED`.
6. Passer les phases en ordre strict et rouvrir les dépendances quand une modification invalide une preuve précédente.
7. Chercher un chemin alternatif légitime en cas de limitation, dans le budget et les permissions. Ne jamais simuler un capteur, outrepasser le consentement ou la sécurité.

Chapitre 2

Objets fondamentaux

2.1 Observation et empreinte

Soit x l'objet, c le contexte et $P = (p_1, \ldots, p_n)$ une famille de sondes. L'observation est

$$O_P(x, c) = \{(p_i, y_i, s_i, m_i, e_i, h_i)\}_{i=1}^{n},$$

où y_i est une vue/valeur ou absence explicite, s_i le statut, m_i les métadonnées, e_i coût/erreur et h_i la provenance. L'original brut, les valeurs décodées et les représentations calculées sont distincts. Une égalité d'empreinte signifie au plus une équivalence relative au contexte et à la famille de sondes ; elle ne prouve pas l'égalité brute des objets.

2.2 Contrat de sonde

Une sonde porte 'id', version, domaine, entrées/sorties, paramètres, dérivation, coût attendu, permissions et vérificateur. Le générateur produit des définitions canoniques uniques ; chaque sortie est hachée et associée à l'objet d'origine. Il contient des primitives, paramètres nouveaux, miroirs, diagnostics et compositions. Les miroirs sont définis par modalité (inversion d'ordre de tokens, géométrie spatiale, temps, dualité de relation) et ne comparent que des sorties compatibles. Une composition déclare les conversions et pertes d'information.

2.3 Distances et collisions

Pour vecteurs valides, $s(a, b) = \max(0, \min(1, \cos(a, b)))$, $d(a, b) = 1 - s(a, b)$. Les objets non vectoriels nécessitent une distance typée documentée. Le hash sert à l'identité exacte et à la provenance, jamais seul à une distance sémantique. Une collision résiduelle regroupe des objets différents pour la tâche mais trop proches sous les sondes ; une fausse séparation éloigne des objets équivalents à la tâche. Les seuils se règlent par expérience ; sans cible légitime, on ne mesure que la cohérence observationnelle.

2.4 Sonde adaptative

La boucle ordonnée est : relire formalisme/checklist/erreurs ; observer ; identifier des déficits/collisions ; proposer des sondes définies et hachées ; ajouter miroirs et compositions compatibles ; exécuter sans cible ; mesurer distances, collisions, stabilité, gain marginal, coût et bruit ; évaluer les labels uniquement après gel des observations ; enregistrer checkpoint et erreurs ; continuer jusqu'à une condition d'acceptation ou à un arrêt honnête. Un budget atteint mène à une reprise possible, pas à une certification. Une candidate de développement n'est pas une compétence générale.

Chapitre 3

RAG fondé sur les espaces sémantiques

3.1 Pipeline canonique

Pour chaque source : préserver original, hash, type et provenance ; décoder avec un adaptateur déclaré ; segmenter en unités avec offsets ; observer chaque unité sous plusieurs sondes ; créer $S_1, \ldots, S_k$; construire pour chaque espace les tokens, relations, statuts, distances et poids ; observer la convergence inter-espaces ; agréger un espace final S^* ; lire les requêtes à travers le même mécanisme ; conserver la provenance finale ; nettoyer les espaces intermédiaires selon politique configurable. L'utilisateur peut inspecter, supprimer, reconstruire ou mettre à jour un espace. Par défaut, la structure finale reste la source de lecture ; une ancienne version est gardée seulement comme backup/audit selon politique.

3.2 Itoken et vtoken

Pour chaque espace S_j, V_j est son vocabulaire et $z_{j,t}$ un vecteur de relations contextuelles du token t. La proximité est $s_j(t,u) = \max(0, \min(1, \cos(z_{j,t}, z_{j,u})))$ pour vecteurs valides, et $d_j(t,u) = 1 - s_j(t,u)$. Les cas sans vecteur restent non applicables plutôt que de recevoir une valeur inventée.

Le statut *itoken* exprime une invariance relative à une famille de sondes, un corpus, une tâche et des seuils ; le statut *vtoken* exprime une variation utile pour distinguer un contexte/sous-région. Aucun token n'est invariant universellement par nom. La stabilité $a(t)$ agrège proportion de vues présentes, stabilité des contextes, persistance de rang et variation d'occurrence ; ses composantes sont gardées séparément.

Soit I_j l'ensemble des itokens et $p_j(t) = \max_{u \in I_j} s_j(t,u)$. Pour un vtoken :

$$\tilde{w}_{j,v}(t) = b_j(t)[\alpha + (1-\alpha)p_j(t)], \qquad 0 < \alpha < 1,$$

où $b_j(t) > 0$ combine fréquence/stabilité normalisées. À base fixe, plus le vtoken est proche d'un itoken, plus son poids ne peut que croître. Pour un itoken, $\bar{p}_j(t)$ est sa proximité moyenne aux autres itokens :

$$\tilde{w}_{j,i}(t) = b_j(t)[\alpha + (1-\alpha)\bar{p}_j(t)].$$

Ces formules sont une règle empirique initiale, versionnée et testable. Elles n'imposent aucun résultat-cible. Les poids gardent les contributions, données, hashes et changements inter-espaces.

3.3 Poids finaux et nombre d'espaces

Le poids final combine stabilité à travers les espaces, proximité aux invariants et utilité à distinguer les contextes ; la fonction d'agrégation et sa version sont enregistrées. Le nombre d'espaces est choisi par gain marginal, changement des ensembles itoken/vtoken, variation de rangs/poids, stabilité du rappel et budget. Il faut plusieurs itérations stables et un test de lecture avant d'annoncer une convergence. Les statuts distinguent `CONVERGED_HEURISTIC_ONLY`, `UNRESOLVED_BUDGET` et validation indépendante. Le nombre de vues n'est pas fixé arbitrairement, ni illimité sans checkpoint.

3.4 Lecteur et contexte de requête

La requête est tokenisée et observée par plusieurs sondes comme les sources. Un lecteur pondéré combine statut itoken/vtoken, poids, distance et proximité contextuelle, tout en gardant références source/offsets, dates, confiance, permissions et lacunes. Un score de passage général est :

$$Score(c \mid q) = \sum_{t \in T(q) \cap T(c)} w_t \, r_{status}(t) \, g(d(t,q), context(t,c)) - \lambda Noise(c),$$

où r_{status} dépend de la classe et où g décroît avec la distance selon une fonction versionnée. Le score ne constitue pas une preuve de vérité. La recherche lexicale classique est un plugin/compétence optionnelle pouvant être choisie ou routée ; elle n'est pas le cœur du RAG.

3.5 Nettoyage et inspection

Chaque espace possède ID, source hashes, version sonde, période, compteurs, distances, poids et état. Une table permet inspection/suppression. Le nettoyage détruit réellement le payload intermédiaire selon la politique, en conservant l'événement de nettoyage et les hashes. Le final stocke hashes des espaces parents et version d'agrégation. La mise à jour suite à une nouvelle source ne doit pas silencieusement rendre les anciens poids incomparables.

Chapitre 4

Compétences et acquisition

4.1 Socle général

Le répertoire des compétences de base comprend création de sondes, observation (auto-observation incluse), création d'empreintes, orchestration/autonomie et raisonnement logique. Un autre outil peut être ajouté au socle seulement s'il est général, transversal, contractuel et vérifié. Les compétences spécialisées sont candidates ou composées, pas « base » par simple étiquette.

4.2 Définition d'une compétence

Un objet compétence contient ID/version, contrats d'entrées/sorties, domaines, invariants supposés, empreintes/sondes utilisées, provenance d'acquisition, paramètres, coûts, effets, permissions, tests de validation/holdout/OOD, replay source-off et état : `BASE`, `CANDIDATE`, `VALIDATED_BOUNDED`, `REVOKED` ou `UNRESOLVED`. La sélection est automatique ou manuelle ; sa justification, ses alternatives et son vérificateur sont conservés.

4.3 Acquisition sans fuite

Les parseurs de dataset gèrent les schémas et clés déclarées. Les sondes voient l'entrée, pas la cible ; les labels ne sont consultés qu'après gel d'observations ou dans la phase d'optimisation supervisée explicitement déclarée. La partition évite les doublons entre train/validation/holdout. Le modèle/capsule est rechargé depuis disque et testé sans l'oracle/source. Le transfert se teste sur entrées inédites/perturbées. Pas de paire input/target artificielle pour du document non supervisé.

4.4 Composition et adaptation

Une compétence peut être arbre, séquence, branche conditionnelle ou exécution parallèle. Chaque nœud/arc a types, préconditions, postconditions, effets, coûts et permissions. Le composite enregistre chaque trace et échoue explicitement si un contrat n'est pas respecté. L'adaptation à un cas inconnu peut tenter une autre compétence, une nouvelle sonde, une composition ou une expérimentation temporaire ; la promotion nécessite des preuves indépendantes.

Chapitre 5

Observateur universel

Chaque adaptateur expose découverte, autorisation, ouverture, lecture, fermeture et état de santé. L'observation des fichiers est non destructive. La caméra et les capteurs physiques n'ont pas d'ouverture automatique sans permission séparée, durée d'accès et indication visibles. Les formats texte, JSON/JSONL, CSV/TSV, Parquet, PDF, DOCX, image, audio et vidéo requièrent des lecteurs spécifiques ; il faut distinguer données brutes, valeurs décodées, caractéristiques structurelles et descriptions/transcriptions dérivées. Un lecteur absent produit `NOT_DECODED` ou `NOT_RUN`. PDF/DOCX/parquet/médias doivent être des plugins explicites avec dépendances vérifiées. L'observation multimodale n'est pas déclarée complète si seuls taille, hash ou dimensions sont disponibles.

Chapitre 6

Mémoire, autonomie, causalité

6.1 Mémoires

Le système sépare source/provenance, épisodes, observations/empreintes, espaces RAG, compétences/capsules, mémoire de raisonnement et journal de contrôle. Chaque événement porte version, temps, hash, source, cause, parent(s), action, effet, score et visibilité. Les données privées sont minimisées ; les événements de contrôle et d'intégrité subsistent même si l'enregistrement détaillé est désactivé.

6.2 Autonomie persistante

À lancement de l'application, la boucle autonome démarre automatiquement et continue tant que l'utilisateur n'arrête pas AAM. À vide, elle inspecte mémoires accessibles, événements, état de compétence et tâches de maintenance autorisées. À l'arrivée d'une requête, elle la priorise et réduit/pose les actions internes gourmandes ; ensuite elle reprend l'agenda. L'autonomie peut focaliser une ou plusieurs mémoires, ajuster les cadences et planifier l'entraînement demandé. Aucun accès réseau/capteur ne découle de l'autonomie seule.

6.3 Causalité des récompenses

Une transition causale contient les champs suivants :

$$e = (\text{id}, \text{parent}, \text{cause}, \text{observation}, \text{action}, \\ \text{verification}, \text{feedback}, \text{reward}, \text{penalty}, \text{pre}, \text{post}, \text{hash})$$

La note utilisateur modifie la priorité de diagnostic mais ne déclare pas une sortie exacte. Un succès vérifié récompense la voie causale qui l'a produit ; une punition suscite observation supplémentaire, changement d'orchestration/compétence et retest. Les modifications temporaires du RAG ont une version et un rollback et sont optionnelles. Le mécanisme de récompense n'est pas modifiable directement par une compétence qui s'auto-évalue.

Chapitre 7

Interface et modes d'exécution

7.1 Chat et historique

Espace de chat type LLM avec historique latéral, conversation nommable, sources et pièces jointes. Le menu « + » propose observer un fichier/objet, sélectionner compétence, focus (plus de sondes/budget/mémoire/cadence), recherche web sous permission. Les paramètres incluent influence auto-observation vers espace sémantique, nombre de sondes auto/manuel, reconstruire/mettre à jour le RAG, sources et focus.

7.2 RAG

Charger formats multiples ; choisir politique ; lancer observation ; inspecter les sources en lecture seule et les espaces comme tableau ; voir le final ; supprimer espaces ; nettoyer intermédiaires ; reconstruire depuis zéro ; mettre à jour après nouvelle source ; tenter extraction de compétences ; afficher les états multimodaux et les échecs de décodage.

7.3 Sondes et compétences

Créer/charger/modifier/versionner/tester des sondes ; régler leur politique, cadence, miroir/composition ; inspecter observations/distances. Voir compétences de base, candidates, validées et composées ; tester, charger, valider ou retirer ; composer un arbre/conditions puis sauvegarder ; sélectionner automatiquement ou manuellement.

7.4 Observation et mémoire

Afficher auto-observation/normal/focus, source, temps, journaux de consultation mémoire, tâches, récompenses/punitions, relations causales, espaces et checkpoints. Le focus peut choisir une source RAG et accroître budget sonde/compute/mémoire sous le plafond fixé.

7.5 Entraînement, Internet et réglages

Lire CSV/JSON/JSONL/Parquet et documents/médias avec champs clés facultatifs lorsque la méthode n'est pas supervisée ; inspection ; ajout au RAG ; exécution RAG ; lancement/pause/arrêt/état d'entraînement ; réseau, recherche web, banque de connecteurs, MCP/API, permissions, sources/citations/dates/licences et sécurité ; réglages généraux versionnés.

7.6 Contrat d'observation multimodale et d'ingestion OCR

Le chemin central `O93Observer.observe_path` appelle les lecteurs bornés de `aam_o93.media_readers` pour les extensions qu'ils prennent effectivement en charge. Le succès d'une empreinte seule

n'est jamais un succès de décodage média. Le hash calculé par le lecteur doit être égal au SHA-256 des octets initiaux ; le contrôleur re-hashe aussi le chemin après lecture et signale tout changement.

— Image : Pillow doit décoder et charger les pixels dans le budget ; les mesures RGB et l'empreinte perceptuelle sont des mesures de bas niveau. L'OCR n'est déclaré que si Tesseract retourne réellement une chaîne non vide.

— Audio : SoundFile/libsndfile est prioritaire, FFmpeg est un fallback réel lorsqu'il est disponible. Le résultat `PCM_DECODED` expose les mesures de signal ; sans moteur ASR configuré, le statut reste `AUDIO_PCM_DECODED_ASR_NOT_CONFIGURED` et aucune transcription n'est fabriquée.

— Vidéo : OpenCV décode un échantillon borné de frames. Un OCR par frame est exposé seulement si exécuté ; la piste sonore reste sans transcription si aucun ASR n'est configuré.

— Dataset : le rapport persistant expose le lecteur, les mesures, les longueurs et hashes des textes OCR, sans stocker le texte OCR brut. `observation_eligible` et `training_eligible` sont séparés ; pixels ou PCM seuls ne deviennent pas artificiellement des exemples textuels.

— RAG : seules les chaînes réellement extraites par OCR image/frames sont transformées en chunks multimodaux, avec provenance du lecteur, moteur OCR, SHA-256 source et drapeaux de contenu non fiable. Aucun prompt, légende ou transcript n'est codé en dur.

— Parquet : l'adaptateur `o930_parquet_flat_fallback` peut lire certains schémas primitifs plats lorsque les moteurs natifs sont absents. Il déclare ses limites et renvoie `NOT_DECODED` pour une fonction non supportée et `ERROR` pour un fichier invalide. Il n'est pas assimilé à un moteur Parquet général.

Les critères de sortie sont limités aux fichiers et budgets réellement testés ; ils ne constituent pas une preuve de compréhension générale des images/sons/vidéos, de performance ASR, ni de disponibilité matérielle ou réseau.

Chapitre 8

Orchestration et pipeline complet

```
BOOT: read formalism, phase checklist, environment/model error ledgers
      verify manifests and workspace ownership; initialize journal/checkpoint
      start autonomy; keep internet and physical sensors opt-in
TASK: create TaskEnvelope(goal, inputs, permission, budget, focus, verifiers)
      observe raw request and all attachments via applicable probes
      generate missing probes, mirrors and compositions; record typed outputs
      update fingerprint graph and compare collisions, invariants and variants
      if knowledge sources exist: build S_1...S_k and final S*; run weighted reader
      optionally route classic lexical retrieval as a separate competence
      choose skill automatic/manual; execute skill or compose new candidate
      independently verify contract, replay/source-off, holdout/OOD where required
      collect user feedback as a signal; do not replace objective verification
      write causal event, results, errors, weights, provenance and checkpoint
      return explicit result/status; resume background self-observation
STOP: pause worker, checkpoint, verify journal, release lock, confirm stop
```

Chapitre 9

Résultats, sécurité et preuve

9.1 Statuts

Les statuts ont des définitions distinctes :

— `PASS_BOUNDED` : les critères strictement énumérés du test ont réussi ; cela ne prouve pas la généralité d'une compétence.

— `CANDIDATE` : objet proposé qui n'est pas encore accepté.

— `REPRODUCTION_ACCEPTED` : reproduction acceptée dans le domaine de validation annoncé.

— `GENERALIZATION_ACCEPTED` : validation indépendante de généralisation sur le domaine déclaré.

— `PARTIAL`, `NOT_RUN` et `NOT_DECODED` : travail partiel, essai non exécuté, ou entrée non décodée.

— `UNRESOLVED_BUDGET`, `ENVIRONMENT_ERROR` et `BLOCKED` : budget insuffisant, problème d'environnement ou précondition bloquante.

— `FAIL_INTEGRITY` et `REVOKED` : intégrité rompue ou acceptation révoquée.

9.2 Sécurité

La sortie RAG est un contenu non fiable ; elle ne peut pas modifier les permissions ni lancer un outil. Le réseau et le matériel sont disabled by default. Les fichiers et chemins sont testés contre symlinks/traversal ; écritures et checkpoints atomiques ; manifestes vérifiables ; capacités de lecture, d'écriture, d'exécution, de capteur et de réseau séparées. L'alternative à une limitation est une autre méthode permise, jamais une permission silencieuse ou un succès simulé.

9.3 Format de rapport obligatoire

Description ; architecture ; pipeline complet ; commande(s) ; environnement/dataset/seed/hashes ; résultats bruts ; interprétation ; conséquences théoriques, conceptuelles, opérationnelles ; erreurs et limites ; conclusion ; statut ; preuve et gate suivante. Les exécutions non faites restent explicitement listées.

Chapitre 10

Phases et critères de sortie

Phase	Nom	Gate de sortie	Dépendances
P0	Audit et base vérifiable	Inventaire/hashes, documents consultés, contrats et statuts, checklists, registre, préflight et tests fondation	–
P1	Observateur universel	Fichiers/valeurs/médias, hash intact, niveaux, statuts de décodage, adaptateurs	P0
P2	Sondes adaptatives	unicité, miroirs, composition, distances, collisions, reprise, pas de fuite	P1
P3	RAG multi-espace	itoken/vtoken/poids, convergence, lecteur, final, nettoyage, tests d'ablation	P2
P4	Compétences	base justifiée, composition, sélection auto/manuelle, replay source-off, transfert	P1–P3
P5	Autonomie	démarrage automatique, focus, causalité, feedback, mémoire, pause/stop/reprise	P0–P4
P6	UI	tous les espaces, historique et contrôles réellement connectés et testés	P1–P5
P7	Dataset/entraînement	formats déclarés, clés conditionnelles, RAG, suivi, pause/stop	P1–P6
P8	Intégration/sécurité	end-to-end, permissions, erreurs, concurrence, non-régression	P1–P7
P9	Paquets de livraison	extraction propre, self-check, hashes, docs et preuves synchronisées	P0–P8
P10	Acceptation externe	OS natif, capteurs réels, réseau configuré, crash/rollback/ACL	P0–P9

Aucune phase suivante ne démarre avant la clôture documentée de toutes les gates actuelles et des dépendances. Toute modification pertinente réouvre les phases impactées ; les anciennes preuves restent historiques.

Chapitre 11

Conséquences et statut

11.1 Théoriques

L'implémentation teste expérimentalement une famille d'empreintes et les transformations entre observations. Les statuts itoken/vtoken et les poids sont empiriques et contextualisés.

11.2 Conceptuelles

La compétence désigne un mécanisme reproductible et adaptable ; RAG, recherche lexicale et génération sont différenciés.

11.3 Opérationnelles

Toutes les actions ont contrats, budgets, permissions, traces et vérifications. L'autonomie est obligatoire en fonctionnement normal mais n'est pas une autorisation d'accès externe.

11.4 Conclusion/statut

Ce formalisme représente la cible normative. Le registre/checklist établit l'état réellement implémenté et testé ; aucune ligne ne peut être déclarée complète sur la seule base de ce document.